

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

LogixPulse Platform — Module 1: CRM & Client Onboarding

IEEE Std 830-1998
Compliant

ISO/IEC/IEEE
29148:2018

Sprint 1
Production
Release

Security
Hardened V2.1

DOCUMENT IDENTIFICATION

LP-SRS-MOD1-2026-V2.1

PUBLICATION DATE

September 25, 2026

PRIMARY AUTHORING SQUADS

Squad PHP-BE, Squad PHP-FE-A, Squad
PHP-FE-B

SUPERVISING LEADERSHIP

David Chen (Technical Head), Practice
Manager

RELEASE VERSION & STATE

Version 2.1 (Consolidated & Approved
Baseline)

TARGET ARCHITECTURE

PHP 8.0+ / PDO / PostgreSQL / MySQL /
Vanilla JS

SQUAD LEADS

Zainab Batool (BE), Fatima Nadeem (FE-
A), Momna Munir (FE-B)

LINE MANAGEMENT OVERSIGHT

Abdullah Bin Masood · Alishba Amin

Executive Summary

The **LogixPulse Platform** is an enterprise-grade, dual-portal web application architected to bridge internal business development operations and external client onboarding workflows into a cohesive, secure ecosystem. Developed during Sprint 1 (Weeks 7 and 8) of the DevLogix Strategic Internship Program (SIP'26) Engineering Track (PHP Practice), Module 1 establishes the operational core of the platform: **CRM & Client Onboarding**.

Historically, enterprise sales teams and onboarding operations suffered from fragmented tooling—leads were tracked in disparate spreadsheets, client contracts required third-party signature tools, and financial documentation was communicated over insecure email channels. LogixPulse eliminates these frictions through two synchronized portals governed by a unified database:

Dual-Portal Architectural Paradigm

1. Internal CRM & Executive Portal

Multi-column visual Kanban board, HTML5 drag-and-drop pipeline progression, debounced multi-field search, real-time KPI stat cards, pipeline velocity metrics, live cross-lead activity feed, and team leaderboard with quota attainment.

2. Unified Core Engine

PHP 8.0+ native session router, centralized PDO singleton with dual-schema compatibility, transactional persistence, bcrypt password hashing, and script-execution-hardened file vaults.

3. External Client Onboarding Portal

Slack-style passwordless 6-digit OTP magic code authentication, self-service password reset, real-time onboarding progression timeline, Master Service Agreement with HTML5 Canvas digital signature pad, and client-isolated invoice downloads.

This Software Requirements Specification (SRS) establishes the definitive technical and behavioral standard for LogixPulse Module 1. It incorporates all requirements from the Week 7-8 Sprint Task Sheets, the Technical Guide for the Head, the Week 7 Day 4 Addendum, the Squad Roster, and architectural incident resolutions documented in the Incident and Prevention Guide.

Table of Contents

1. Introduction & Project Scope	Page 3
2. Overall System Description & Topology	Page 4
3. User Classes, Personas & Role-Based Access Control (RBAC)	Page 5
4. System Features & Functional Requirements (FRs)	Page 6
4.1 Subsystem 1: Dual-Portal Authentication & Identity Governance	Page 6
4.2 Subsystem 2: Internal CRM Pipeline & Kanban Management	Page 8
4.3 Subsystem 3: Activity Logging & Historical Audit Trail	Page 10
4.4 Subsystem 4: Executive Business Intelligence & Analytics	Page 11
4.5 Subsystem 5: External Client Portal & Onboarding Progression	Page 13
4.6 Subsystem 6: Digital Master Agreement & E-Signature Engine	Page 14
4.7 Subsystem 7: Secure Invoicing & Document Repository	Page 15
5. External Interface Specifications & REST API Dictionary	Page 16
6. Database Architecture & Complete 13-Table Schema Dictionary	Page 19
7. Non-Functional Requirements (Security, Performance, Usability)	Page 22
8. Verification Protocol, Acceptance Criteria & Traceability Matrix	Page 24

9. Sprint Governance, Squad Rosters & Incident Retrospective	Page 26
10. Appendices & Formal Sign-Off	Page 28

1. Introduction & Project Scope

1.1 Purpose of the Document

This Software Requirements Specification (SRS) establishes the complete functional, technical, interface, and behavioral specifications for **LogixPulse Module 1: CRM & Client Onboarding**. It serves as the primary contractual and architectural baseline for software engineers, QA analysts, DevOps practitioners, and executive leadership at DevLogix Technology. Every functional requirement specified herein is directly mapped to individual sprint deliverables executed across Weeks 7 and 8.

1.2 Document Conventions & RFC 2119 Compliance

This document adheres to the terminology conventions defined in **RFC 2119**:

- **MUST / SHALL / REQUIRED:** An absolute engineering requirement. Non-compliance results in immediate rejection of the build or task during daily gate evaluations.
- **SHOULD / RECOMMENDED:** Strong best practice. Exceptions must be documented and approved by the Technical Head.
- **MAY / OPTIONAL:** Discretionary feature or additive polish that does not block release milestones.

1.3 Intended Audience

Target Role	Primary Interest	Key Sections of Focus
Backend Engineers (Squad BE)	API schemas, PDO prepared statements, database schemas, RBAC session validation, storage isolation.	Section 4.1, 4.3, 4.7, Section 5 (APIs), Section 6 (Database), Section 7 (Security).
Frontend Engineers (Squads FE-A / FE-B)	UI components, HTML5 drag-and-drop, Canvas signature rendering, fetch integration, error handling.	Section 4.2, 4.4, 4.5, 4.6, Section 5 (API contracts), Section 7 (Usability).
QA & Squad Leads	Daily acceptance criteria, cross-squad testing matrices, demo exit gates, defect logging.	Section 4 (Done When clauses), Section 8 (Verification & RTM), Section 9.
Practice Management & Head	Delivery compliance, security posture, governance rules, sprint exit criteria.	Executive Summary, Section 3 (RBAC), Section 8, Section 9 (Governance).

1.4 Product Scope & Core Value Proposition

Module 1 encompasses the foundational operational capabilities required to acquire, track, convert, contract, and invoice enterprise clients. The module guarantees:

1. **Zero Data Leakage Across Roles:** Clients never access internal CRM leads or administrative endpoints. Clients strictly download only invoices tied to their primary session identifier.
2. **Pure Native Stack Simplicity:** Elimination of cumbersome monolithic frameworks (e.g., Laravel, Symfony) in favor of ultra-performant, auditable PHP 8.0+ with PDO, standard SQL, and Vanilla JavaScript.
3. **Full Lifecycle Traceability:** Every stage movement, manual note, magic code issuance, and agreement signature is permanently recorded in structured audit logs.

1.5 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
ARR	Annual Recurring Revenue. Metric tracked in high-value lead opportunities and activity feeds.
Base64	Binary-to-text encoding scheme utilized to transfer HTML5 Canvas digital signatures to the backend.
Bcrypt	Adaptive cryptographic hash function implemented via PHP's native <code>password_hash()</code> .
BOLA / IDOR	Broken Object Level Authorization / Insecure Direct Object Reference. Vulnerability mitigated by session-bound client isolation.
Kanban	Visual agile workflow method featuring 4 progressive stages: New, Contacted, Meeting Booked, Closed.
Magic Code	Cryptographically randomized 6-digit numeric one-time password (OTP) used for passwordless client login.
MSA	Master Service Agreement. Legally binding document reviewed and digitally signed within the client portal.
MTD	Month-To-Date. Aggregation period utilized in executive KPI cards and quota attainment calculations.
PDO	PHP Data Objects. Database abstraction layer providing parameterized prepared statements.
RBAC	Role-Based Access Control. Security model enforcing authorization checks against user roles.
RoE	Rules of Engagement. Mandatory behavioral and technical non-negotiables governing sprint execution.

2. Overall System Description & Topology

2.1 Product Perspective & Context

LogixPulse operates as an on-premise or cloud-hosted web system deployed across enterprise infrastructure. Built to run cleanly on standard Linux (Ubuntu/Debian) or Windows Server runtimes with PHP 8.0+ and PostgreSQL 14+ (or MySQL 5.7+ / MariaDB 10.3+), it requires zero heavy runtime dependencies or build steps.

2.2 Multi-Tier System Architecture

The system implements a modular multi-tier architecture partitioned into distinct, decoupled operational layers:

LogixPulse Multi-Tier Architecture Diagram



2.3 Operating Environment & Infrastructure Boundaries

- **Server Runtime:** PHP 8.0.0 or higher. Mandatory extensions: `pdo` , `pdo_pgsql` or `pdo_mysql` , `fileinfo` , `mbstring` , `json` .

-
- **Database Engine:** PostgreSQL 14+ (hosted on remote production VPS) or MySQL 5.7+ / MariaDB 10.3+ (local development standard).
 - **Web Servers:** Apache 2.4+ (with `mod_rewrite` and `AllowOverride All` for `.htaccess` protection) or Nginx with PHP-FPM.
 - **Client Browsers:** Evergreen modern browsers (Chrome 100+, Edge 100+, Firefox 100+, Safari 15+). Zero Internet Explorer support required.
 - **Responsive Viewports:** Desktop (1920×1080, 1440×900), Tablet (768×1024), and Mobile (375×667 down to 320×568).

3. User Classes, Personas & Role-Based Access Control (RBAC)

3.1 User Class Descriptions

LogixPulse enforces strict Role-Based Access Control (RBAC). A user's identity is authenticated server-side and maintained in the native PHP session array (`$_SESSION['user_id']` , `$_SESSION['role']`). The system recognizes five operational personas:

1. **System Administrator (`admin`)**: Superuser responsible for platform governance. Has unrestricted privileges across all CRM leads, internal user management, invoice uploading, and deletion actions.
2. **Strategic Executive (`executive`)**: Executive stakeholder requiring read-only macro visibility into business development performance, revenue velocity, KPI cards, and cross-team activity feeds.
3. **Practice Head (`head`)**: Engineering or sales leader supervising squads and pipeline velocity. Can view, manage, and assign leads across all territories.
4. **Business Development Representative (`team` / `staff` / `user`)**: Frontline sales agent. Manages assigned leads, executes stage transitions on the Kanban board, adds manual communication notes, and tracks individual quota attainment.
5. **Client (`client`)**: External commercial partner. Restricted exclusively to the Client Onboarding Portal. Can review project milestones, download personal invoices, and execute legal agreements.

3.2 Master RBAC Permissions Matrix

System Feature / API Endpoint	Administrator (admin)	Executive (executive)	BD Head (head)	BD Rep (team)	Client (client)
Access CRM Dashboard & Kanban	FULL	READ	FULL	FULL	BLOCKED (403)
Create New Lead (save_lead.php)	ALLOWED	DENIED	ALLOWED	ALLOWED	BLOCKED (403)
Move Lead Stage (update_lead_stage.php)	ALLOWED	DENIED	ALLOWED	ALLOWED	BLOCKED (403)
Add Lead Manual Note (add_note.php)	ALLOWED	DENIED	ALLOWED	ALLOWED	BLOCKED (403)
Delete Lead (leads_delete.php)	ALLOWED	DENIED	DENIED	DENIED	BLOCKED (403)
View Executive KPIs & Velocity	ALLOWED	ALLOWED	ALLOWED	ALLOWED	BLOCKED (403)
Upload Client Invoices (upload.php)	ALLOWED	DENIED	ALLOWED	DENIED	BLOCKED (403)
Download Invoices (download.php)	ANY INVOICE	DENIED	ANY INVOICE	DENIED	OWN ONLY
View Client Timeline & Progress	READ ALL	DENIED	READ ALL	READ ALL	OWN ONLY
Sign Legal Master Agreement (MSA)	DENIED	DENIED	DENIED	DENIED	ALLOWED

CRITICAL SECURITY RULE: Zero-Trust Client Isolation

Under no circumstances may a user authenticated with role `client` receive access to another client's invoice, contract, or timeline data. `backend/download.php` and `includes/timeline.php` MUST extract the tenant identifier strictly from `$_SESSION['user_id']` and NEVER from HTTP request parameters (`$_GET['client_id']` or `$_POST['client_id']`).

4. System Features & Functional Requirements

4.1 Subsystem 1: Dual-Portal Authentication & Identity Governance

FR-AUTH-01	Traditional Credential Authentication	ALL USERS
Description: Authenticate users using corporate email and plaintext password validated against native Bcrypt cryptographic hashes stored in the <code>users</code> table. Trigger: User submits traditional login form on <code>index.html</code> or <code>backend/internal_login.php</code>. Flow of Events: - Browser sends <code>POST /api/login.php</code> with <code>email</code> and <code>password</code>. - Backend sanitizes inputs and executes parameterized query: <code>SELECT * FROM users WHERE email = ? LIMIT 1</code>. - If user exists, backend evaluates password using <code>password_verify(\$password, \$user['password_hash'])</code>. - On success: calls <code>session_regenerate_id(true)</code>, initializes <code>\$_SESSION['user_id']</code>, <code>\$_SESSION['email']</code>, <code>\$_SESSION['role']</code>. - Returns JSON: <code>{"error": false, "role": "client", "redirect": "client-portal-dashboard/index.php"}</code>. - On failure: returns HTTP 401 with JSON: <code>{"error": true, "message": "Invalid email or password"}</code>. Done When: Postman and browser testing with valid credentials returns successful session and correct role; testing with invalid credentials returns "login failed" without exposing database errors or stack traces.		
FR-AUTH-02	Passwordless Magic Code (OTP) Authentication	CLIENT ONLY
Description: Enables Slack-style passwordless authentication. A client enters their registered email, receives a cryptographically generated 6-digit numeric OTP valid for 10 minutes, and authenticates by submitting the code. Endpoints: <code>POST /api/request_magic_code.php</code>, <code>POST /api/verify_magic_code.php</code>. Business Rules: - Code must be exactly 6 random digits generated via <code>random_int(100000, 999999)</code>. - Expiration period: exactly 10 minutes (<code>NOW() + INTERVAL '10 MINUTE'</code>). - Brute-force protection: each code record tracks <code>attempts</code>. If attempts exceed 5, the code is invalidated. - Replay protection: once verified, <code>used_at</code> is set to the current timestamp. Single-use only. Done When: Client requests code → code is saved in <code>magic_codes</code> → submitting code within 10 minutes logs client in → submitting expired code, wrong code, or already-used code rejects with specific error message.		

FR-AUTH-03 Self-Service Password Reset Flow**ALL USERS**

Description: Two-step password recovery allowing users to reset their forgotten password via secure cryptographic token.

Endpoints: `POST /api/request_password_reset.php` , `POST /api/reset_password.php` .

Flow of Events:

1. User enters email on `reset-password.html` .
2. Backend generates 64-character random hex token via `bin2hex(random_bytes(32))` , computes SHA-256 hash (`hash('sha256', $token)`), and stores hash in `password_resets` with 1-hour expiration.
3. User navigates to `set-new-password.html?token=...` and submits new password (minimum 8 characters).
4. Backend hashes input with `password_hash($newPassword, PASSWORD_DEFAULT)` , updates `users.password_hash` , and marks token as used.

Done When: User can complete reset flow → immediately log in with new password → old password no longer works → token cannot be reused.

FR-AUTH-04 Session Identity Verification & Router**SYSTEM WIDE**

Description: Provides heartbeat identity checks and automated traffic routing based on active session role.

Behaviors:

- `GET /index.php` : If unauthenticated, serves/redirects to `index.html` . If authenticated as `client` , redirects to `client-portal-dashboard/index.php` . If authenticated as `admin / staff` , redirects to `backend/main_dashboard.php` .
- `GET /api/who_is_logged_in.php` : Returns active session user metadata: `{"logged_in": true, "user_id": 1, "role": "admin", "email": "admin@logixpulse.com"}` .
- `GET /logout.php` : Calls `session_destroy()` , unsets `$_SESSION` , purges session cookies, and redirects to `index.html` .

Done When: Opening the root domain correctly routes users according to their active session; logging out fully terminates session state across both portals.

4.2 Subsystem 2: Internal CRM Pipeline & Kanban Management

FR-CRM-01	Multi-Column Visual Kanban Board	STAFF / BD / ADMIN
Description: Displays all active commercial leads partitioned into exactly four horizontal pipeline columns: New, Contacted, Meeting Booked, and Closed. Data Integration: On page initialization, queries <code>GET /api/grouped-leads.php</code>. The backend invokes <code>LeadRepository::getGroupedLeads()</code> to retrieve all leads grouped by status/stage. Card Metadata: Each lead card displays: Lead Name, Company, Primary Email, Phone Number, Source Tag (e.g., Inbound, LinkedIn, Referral), and Assigned Representative avatar. Done When: All four columns render side-by-side with equal proportional width; live database leads populate their respective columns without layout corruption across screen resizes.		
FR-CRM-02	Interactive Drag-and-Drop Stage Transitions	STAFF / BD / ADMIN
Description: Users can click and drag any lead card from its current column and drop it into a target column using the native HTML5 Drag and Drop API. Client-Side Implementation: Cards specify <code>draggable="true"</code> with event listeners for <code>dragstart</code>, <code>dragover</code>, <code>dragleave</code>, and <code>drop</code>. Ghost previews and hover column highlights provide visual feedback. Persistence Synchronization: Upon <code>drop</code>, frontend immediately dispatches <code>POST /api/update_lead_stage.php</code> containing <code>lead_id</code> and <code>new_stage</code>. If the API returns success, a toast confirmation appears. If the API fails, the card reverts to its origin column with an error toast. Done When: Dragging a card to a new column moves it visually, persists to the database, and the new position survives a complete browser hard refresh.		
FR-CRM-03	Atomic Stage Update & Transactional Audit Logging	BACKEND ENGINE
Description: Backend processor for stage changes enforcing database transaction integrity. Endpoint: <code>POST /api/update_lead_stage.php</code>. Execution Logic: <pre>\$pdo->beginTransaction(); // 1. Dual-column compatibility update \$stmt = \$pdo->prepare("UPDATE leads SET status = ?, stage = ?, updated_at = NOW() WHERE id = ?"); \$stmt->execute([\$newStage, \$newStage, \$leadId]); // 2. Automated audit log insertion \$logStmt = \$pdo->prepare("INSERT INTO activity_logs (lead_id, user_id, action, activity_type, description, note, created_at) VALUES (?, ?, 'stage_change', 'stage_change', ?, ?, ?)"); \$desc = "Moved from {\$oldStage} to {\$newStage}"; \$logStmt->execute([\$leadId, \$_SESSION['user_id'], \$desc, \$desc, \$_SESSION['name']]); \$pdo->commit();</pre> Done When: Executing a stage change modifies both <code>status</code> and <code>stage</code> columns in <code>leads</code> AND creates a corresponding entry in <code>activity_logs</code> within a single atomic commit.		

FR-CRM-04

Add Lead Modal & Server-Side Validation

STAFF / BD / ADMIN

Description: Modal form allowing BD representatives to capture new inbound prospects directly onto the CRM board.

Input Fields: Full Name (required), Email Address (required, valid email syntax), Phone Number (required), Acquisition Source (select: Website, LinkedIn, Referral, Cold Outreach).

Server-Side Validation: `POST /api/save_lead.php` validates that required fields are non-empty and verifies email structure with `filter_var($email, FILTER_VALIDATE_EMAIL)`. Rejects invalid submissions with HTTP 422.

Board Insertion: Upon successful insert, returns newly created lead JSON. Frontend dynamically injects the card into the **New** column without reloading the page.

Done When: Submitting valid lead details creates record in `leads`, logs creation in `activity_logs`, and card instantly appears in the New column; invalid email formats are rejected with a clear error toast.

FR-CRM-05

Debounced Multi-Column Lead Search

STAFF / BD / ADMIN

Description: Real-time search bar embedded in the CRM board header that filters displayed leads as the user types.

Query Logic: Client debounces input by 300ms, then invokes `GET /api/lead_search.php?search={query}`. The backend executes parameterized SQL with wildcards: `SELECT * FROM leads WHERE name LIKE ? OR email LIKE ? OR phone LIKE ? OR company LIKE ?`.

Edge Cases: Submitting an empty query returns all active leads. If zero matches are found, columns display an empty-state message: "No matching leads found."

Done When: Typing partial strings (e.g., "Ah" for "Ahmed") immediately narrows board cards to matching records across name, email, or company; clearing the search restores the full board.

FR-CRM-06

Lead Profile & Details Sliding Drawer

STAFF / BD / ADMIN

Description: Clicking any lead card triggers an animated offcanvas slide-over drawer displaying full customer information, current stage badge, acquisition metadata, and complete historical audit trail.

Data Endpoints: `GET /api/lead_details.php?id={id}` for profile attributes and `GET /api/lead_activity.php?lead_id={id}` for audit history.

Done When: Clicking different cards dynamically populates the drawer with the exact attributes and timeline of the selected lead, with zero cross-lead data bleed.

FR-CRM-07

Administrative Lead Deletion with Two-Step Confirmation

ADMIN ONLY

Description: Protects the database from accidental lead loss. Only users with role `admin` can delete a lead. Triggering delete presents a modal confirmation: "Are you sure you want to permanently delete this lead? This action cannot be undone."

Security Check: `POST /api/leads_delete.php` checks `$_SESSION['role'] === 'admin'`. Rejects any non-admin role with HTTP 403 Forbidden.

Done When: Admin can delete lead after confirming modal → card vanishes from board → non-admin users attempting delete are rejected with "Not allowed".

4.3 Subsystem 3: Activity Logging & Historical Audit Trail

FR-LOG-01	Manual Free-Text Note Capture	STAFF / BD / ADMIN
Description: Enables BD agents to log unstructured qualitative updates (e.g., "Client requested revised quote with 10% volume discount") within the lead details drawer. Endpoint: POST /api/add_note.php . Accepts lead_id and note . Persistence Schema: Writes to activity_logs with activity_type = 'note' , action = 'add_note' , note = \$noteText , created_by = \$_SESSION['name'] , and created_at = NOW() . Done When: Submitting a note immediately injects it at the top of the lead's activity history drawer with author attribution and relative timestamp ("Just now").		
FR-LOG-02	Chronological Lead Activity Stream	STAFF / BD / ADMIN
Description: Streams the comprehensive chronological audit history of a specific lead, interleaving automatic system events (stage movements, creation) and manual agent notes. Endpoint: GET /api/lead_activity.php?lead_id={id} . Sorting: Strictly ordered descending: ORDER BY created_at DESC, id DESC . Done When: Opening any lead drawer shows newest events first; performing a stage drag or adding a note immediately reflects at the top of the list.		
FR-LOG-03	System-Wide Live Recent Activity Feed	EXECUTIVE / HEAD / ADMIN
Description: Live feed card on the Executive CRM Dashboard summarizing cross-entity activity across all company leads, client agreements, and invoices. Endpoint: GET /api/activity_feed.php (Task BE-W8D2-4) . Visual Card Elements: Actor avatar initials, actor name, action description, highlighted commercial details (e.g., "\$45,000 ARR", agreement signed), relative timestamp, pulsating green "Live" indicator, and "Open Full Audit" action button. Done When: Moving a lead or adding a note on the Kanban board causes a new item to appear at the top of the executive activity panel without a manual page refresh.		

4.4 Subsystem 4: Executive Business Intelligence & Analytics

FR-KPI-01

Executive Dashboard KPI Stat Cards

EXECUTIVE / HEAD / ADMIN

Description: Row of four top-level executive KPI metric cards situated at the pinnacle of the CRM dashboard header, providing real-time macro operational visibility.

Endpoint: GET /api/dashboard_kpi.php (Tasks BE-W7D5-4 & FEA-W7D5-4).

Card Metric Specifications:

KPI Title	Underlying Database Query Calculation	Trend Delta Calculation
Total Leads	COUNT(id) FROM leads	Percentage delta vs equivalent prior 30-day window.
Active Deals	COUNT(id) FROM leads WHERE status IN ('contacted', 'meeting')	Percentage delta vs previous calendar week.
Meetings This Week	COUNT(id) FROM activity_logs WHERE action = 'stage_change' AND description LIKE '%Meeting Booked%' AND created_at >= DATE_TRUNC('week', NOW())	Comparison against meetings booked in prior week.
Closed Deals (MTD)	COUNT(id) FROM leads WHERE status = 'closed' AND updated_at >= DATE_TRUNC('month', NOW())	Comparison against closed count on same calendar day last month.

Done When: All four stat cards display real database numbers, colored trend arrows (±%), and progress bars that precisely match manual SQL aggregation counts.

FR-CRM-02

Pipeline Overview & Velocity Analytics

EXECUTIVE / HEAD / ADMIN

Description: Detailed executive panel analyzing pipeline conversion efficiency and capital volume across stages.

Endpoint: GET /api/pipeline_velocity.php (Tasks BE-W8D1-4 & FEA-W8D1-4).

Analytical Components:

- Stage Rows:** Displays account count, total deal valuation (\$), and conversion rate (%) from the previous stage for each canonical stage.
- Conversion Progress Bars:** Colored progress indicators sized proportionately to stage conversion efficiency.
- Average Deal Velocity:** Computes average duration in elapsed days from leads.created_at to stage transition to closed .
- Top Acquisition Source:** Identifies source category with highest conversion volume (e.g., "Inbound Organic - 42%").
- By Value / By Volume Toggle:** Interactive switch toggling visual emphasis between dollar valuation and account count without requiring an API roundtrip.

Done When: The section displays real stage conversions matching pgAdmin/MySQL queries; toggling Value/Volume changes emphasis cleanly without console errors.

FR-CRM-03	Sales Team Leaderboard & Quota Engine	EXECUTIVE / HEAD / ADMIN
Description: High-resolution leaderboard tracking representative sales performance, deal throughput, and quota attainment. Endpoint: GET /api/team_leaderboard.php (Tasks BE-W8D3-4 & FEA-W8D3-4). Table Columns: Representative Name & Avatar, Active Leads Count, Meetings Held, Deals Won, Total Won Valuation (\$), Quota Attainment Badge (<70% red, 70-99% amber, ≥100% green), and "View Pipeline" deep link. Controls & Filtering: Territory dropdown filter (North America, EMEA, APAC), Tab toggle (Representative Performance vs High-Value Opportunities), and pagination controls ("Showing X of Y reps"). Done When: Leaderboard reflects live representative statistics; territory filter accurately narrows table records; quota badges display proper color thresholds.		
FR-CRM-04	Executive Report Export Engine	EXECUTIVE / HEAD / ADMIN
Description: Allows executive leadership to export active KPI statistics, pipeline overview figures, and leaderboard standings into a clean printable report or structured document. Implementation: "Export Report" button generates a sanitized print view via CSS print media styles or initiates client-side data export containing current dashboard metrics. Done When: Clicking "Export Report" generates a complete printable summary containing current live numbers matching the screen display.		

4.5 Subsystem 5: External Client Portal & Onboarding Progression

FR-CLT-01	Interactive Client Onboarding Milestone Timeline	CLIENT ONLY
Description: Visual step-by-step progress timeline displaying the client's current onboarding lifecycle stage (e.g., Discovery → Agreement Signed → Technical Architecture → Development → Delivery). Data Schema: Driven by tables <code>client_timeline_progress</code> and <code>timeline_steps</code>. Dynamic State Resolution: Backend function <code>getClientTimeline(\$clientId)</code> evaluates each step against <code>current_step_order</code>: <code>step_order < current_step_order</code> → Status = Completed (green checkmark, clickable). <code>step_order == current_step_order</code> → Status = Current / In Progress (pulsating active blue badge). <code>step_order > current_step_order</code> → Status = Upcoming / Pending (muted gray icon, disabled). Security & Isolation: <code>\$clientId</code> is pulled strictly from <code>\$_SESSION['user_id']</code>. Client A can never view Client B's progress. Done When: Logging in as two distinct sample clients displays two distinct onboarding phases matching database seed rows; unauthenticated access displays an invitation to log in.		
FR-CLT-02	Client Engagement Overview & Active Projects	CLIENT ONLY
Description: Dashboard summary cards presenting active commercial projects, assigned account manager contact information, and primary engagement deliverables. Data Source: Evaluates <code>projects</code> table where <code>projects.client_id = \$_SESSION['user_id']</code>. Done When: The client dashboard accurately reflects projects assigned to the logged-in user without cross-tenant leakage.		

4.6 Subsystem 6: Digital Master Agreement & E-Signature Engine

FR-AGR-01	Master Service Agreement (MSA) Legal Shell	CLIENT ONLY
Description: Renders the formal legal contract text on <code>agreement_page/index.html</code>, formatted with standard commercial clauses: Scope of Work, Intellectual Property, Payment Terms, Confidentiality, and Governing Law. Status Awareness: Queries <code>agreements</code> table. If the agreement is already signed, displays the stored signature PNG, the signing timestamp, and locks the agreement against re-signing. Done When: Terms and conditions render crisply on both desktop and mobile viewports; signed agreements display read-only signed status.		

FR-AGR-02

HTML5 Canvas Digital Signature Pad

CLIENT ONLY

Description: Interactive drawing surface allowing clients to draw their legal handwritten signature using mouse, stylus, or mobile touch screen.

Technical Canvas Specifications:

- Implemented via HTML5 `<canvas id="signature-pad">`.
- Captures `mousedown`, `mousemove`, `mouseup` for desktop pointer input.
- Captures `touchstart`, `touchmove`, `touchend` with `e.preventDefault()` to prevent mobile page scrolling while signing.
- Smoothing: Uses 2D context line smoothing (`lineCap = "round"`, `lineJoin = "round"`, `lineWidth = 2.5`).
- Action Controls: "Clear Signature" button resets canvas (`ctx.clearRect(0, 0, width, height)`); "Accept & Sign Agreement" serializes drawing.

Done When: User can draw smooth continuous strokes with mouse or touch; clicking "Clear" wipes the canvas; empty canvas submissions are rejected with validation prompt.

FR-AGR-03

Cryptographic Signature Ingestion & File Storage

BACKEND ENGINE

Description: Backend processor that ingests serialized canvas vector data, decodes binary image, and persists signature to disk.

Processing Pipeline:

1. Client JS executes `canvas.toDataURL("image/png")` producing Base64 data URI string.
2. Dispatches `POST /agreement_page/save_signature.php` with JSON body: `{"agreement_id": 1, "signature": "..."} .`
3. Backend strips MIME prefix, decodes via `base64_decode()`, and generates unique randomized filename: `sig_<clientId>_<timestamp>_<randomHash>.png`.
4. Writes file to hardened storage directory: `uploads/signatures/`.
5. Executes SQL transaction updating `agreements`: sets `status = 'signed'`, `signed_date = NOW()`.
6. Appends audit record into `activity_logs`.

Done When: Signing an agreement writes a genuine PNG image to `uploads/signatures/`, marks agreement signed in database, and renders success toast.

4.7 Subsystem 7: Secure Invoicing & Document Repository

FR-INV-01

Administrative Protected Invoice Upload

ADMIN ONLY

Description: Administrative interface on `backend/upload.php` permitting staff to upload commercial billing invoices assigned to specific client accounts.

Rigorous File Validation Pipeline:

- **Role Verification:** Rejects any user without `$_SESSION['role'] === 'admin'`.
- **Extension Whitelist:** Strictly inspects filename extension; permits **PDF only** (`.pdf`). Any `.php`, `.phtml`, `.exe`, or `.sh` is blocked immediately.
- **MIME Inspection:** Uses PHP's `finfo_file(FILEINFO_MIME_TYPE)` to verify real file MIME is `application/pdf`.
- **File Size Ceiling:** Rejects files exceeding 10 Megabytes (10,485,760 bytes).
- **Path Sanitization & Name Randomization:** Never stores original filename on disk. Generates randomized UUID storage name: `inv_<clientId>_<bin2hex(random_bytes(16))>.pdf`.
- **Database Record:** Inserts metadata into `invoices` table storing original filename, stored filename, `client_id`, amount, and issue date.

Done When: Admin can upload valid PDF resulting in new `invoices` row and disk file; non-admin users, non-PDF files, and oversized files are rejected with clean error responses.

FR-INV-02

Client-Isolated Secure Invoice Streaming

CLIENT / ADMIN

Description: Secure document streaming proxy preventing unauthorized direct file access or Insecure Direct Object References (IDOR).

Endpoint: `GET /backend/download.php?id={invoice_id}`.

Authorization & Streaming Protocol:

```
// 1. Session enforcement
if (!isset($_SESSION['user_id'])) { http_response_code(401); die("Not authenticated"); }

// 2. Fetch invoice metadata via PDO
$stmt = $pdo->prepare("SELECT * FROM invoices WHERE id = ?");
$stmt->execute([$invoiceId]);
$inv = $stmt->fetch(PDO::FETCH_ASSOC);
if (!$inv) { http_response_code(404); die("Invoice not found"); }

// 3. Strict Tenant Isolation
if ($_SESSION['role'] === 'client' && (int)$inv['client_id'] !== (int)$_SESSION['user_id']) {
    http_response_code(403);
    die("Access Denied: You do not own this document.");
}

// 4. Safe binary file streaming
$filePath = __DIR__ . '/../uploads/invoices/' . basename($inv['stored_filename']);
header('Content-Type: application/pdf');
header('Content-Disposition: inline; filename="' . addslashes($inv['original_filename']) . '"');
header('Content-Length: ' . filesize($filePath));
readfile($filePath);
exit;
```

Done When: Client A can download Client A's invoice; attempting to download Client B's invoice returns HTTP 403 Forbidden; direct browser access to `uploads/` is blocked.

FR-INV-03	Client Invoice Listing & Status Table	CLIENT ONLY
Description: Invoices tab on the client dashboard displaying all commercial invoices billed to the client's account. Table Attributes: Invoice Number, Issue Date, Due Date, Billed Amount (\$), Status Badge (Paid, Due, Pending), and "Download PDF" action button. Empty State Handling: If no invoices exist for the account, renders a friendly empty-state illustration with message: "No invoices issued yet. Your billing documents will appear here." Done When: Real client invoices populate the table; clicking "Download" opens the specific verified PDF file; accounts with zero invoices render clean empty state without breaking layout.		
FR-INV-04	Storage Subsystem Hardening & Script Execution Block	SECURITY INFRA
Description: Prevents Remote Code Execution (RCE) attacks by enforcing web server execution restrictions inside the upload directory. Configuration: An immutable <code>.htaccess</code> file is deployed in <code>uploads/</code> containing: <pre># Block direct script execution in uploads vault <FilesMatch ".(php phtml php3 php4 php5 php7 phps cgi pl sh exe)\$"> Order Deny,Allow Deny from all </FilesMatch> Options -Indexes -ExecCGI</pre> Done When: Attempting to directly request or execute any PHP file placed in <code>uploads/</code> yields HTTP 403 Forbidden.		

5. External Interface Specifications & REST API Dictionary

5.1 Communication Protocols & General Conventions

LogixPulse APIs adhere to RESTful conventions communicating over standard HTTP/HTTPS. Unless otherwise specified (e.g., binary PDF streaming or multi-part uploads), all endpoints accept and return JSON payloads formatted with standard HTTP response status codes:

- **200 OK** : Request completed successfully.
- **400 Bad Request** : Missing required fields or malformed payload.
- **401 Unauthorized** : Missing or expired session; user must authenticate.
- **403 Forbidden** : Authenticated user lacks RBAC role permissions for this resource.
- **404 Not Found** : Specified entity (lead, invoice, user) does not exist.
- **422 Unprocessable Entity** : Validation failed (e.g., invalid email syntax).
- **500 Internal Server Error** : Caught backend exception. Always returns clean JSON: `{"error": true, "message": "Database error occurred"}`.

5.2 Master API Endpoint Catalog

Method	Endpoint URI	Auth Required	Request Parameters / Payload	Response Payload Summary
POST	/api/login.php	None (Public)	email , password (Form / JSON)	{ "error": false, "role": "admin", "redirect": "..."}
GET	/api/who_is_logged_in.php	Session	None	{ "logged_in": true, "user_id": 1, "role": "admin"}
GET	/logout.php	Session	None	Redirect to index.html (Session destroyed)
POST	/api/request_magic_code.php	None (Public)	{ "email": "client@example.com"}	{ "error": false, "message": "Code sent to email"}
POST	/api/verify_magic_code.php	None (Public)	{ "email": "...", "code": "123456"}	{ "error": false, "role": "client", "redirect": "..."}
POST	/api/request_password_reset.php	None (Public)	{ "email": "user@example.com"}	{ "error": false, "message": "Reset token generated"}
POST	/api/reset_password.php	None (Public)	{ "token": "...", "password": "..."}	{ "error": false, "message": "Password updated"}
GET	/api/grouped-leads.php	Staff / Admin	None	{ "new": [...], "contacted": [...], "meeting": [...], "closed": [...]}
POST	/api/save_lead.php	Staff / Admin	{ "name": "..", "email": "..", "phone": "..", "source": ".."}	{ "error": false, "lead_id": 42, "lead": {...}}
POST	/api/update_lead_stage.php	Staff / Admin	{ "lead_id": 42, "stage": "contacted"}	{ "error": false, "message": "Stage updated"}
GET	/api/lead_search.php	Staff / Admin	Query param: ?search={query}	[{ "id": 1, "name": "...", "company": "..."}, ...]
GET	/api/lead_details.php	Staff / Admin	Query param: ?id={leadId}	{ "id": 42, "name": "...", "email": "...", "stage": "..."}
GET	/api/lead_activity.php	Staff / Admin	Query param: ?lead_id={leadId}	[{ "id": 10, "action": "..", "note": "...",

Method	Endpoint URI	Auth Required	Request Parameters / Payload	Response Payload Summary
				"created_at": ".."}, ...]
POST	/api/add_note.php	Staff / Admin	{"lead_id": 42, "note": "Meeting confirmed"}	{"error": false, "message": "Note saved successfully"}
GET	/api/dashboard_kpi.php	Executive / Staff	None	{"total_leads": 120, "active_deals": 35, "meetings": 14, "closed": 8}
GET	/api/pipeline_velocity.php	Executive / Staff	None	{"stages": [...], "avg_velocity_days": 18.4, "top_source": "Inbound"}
GET	/api/activity_feed.php	Executive / Staff	Query param: ?limit=15	[{"actor": "Sarah M", "action": "stage_move", "timestamp": ".."}, ...]
GET	/api/team_leaderboard.php	Executive / Staff	Query param: ?territory=all	[{"rep": "Alice N", "active": 12, "quota_percent": 115}, ...]
POST	/backend/upload.php	Admin Only	Multipart: file (PDF), client_id, amount	{"error": false, "invoice_id": 15, "filename": "..."} Binary Stream: Content-Type: application/pdf
GET	/backend/download.php	Session (Isolated)	Query param: ?id={invoiceId}	
POST	/agreement_page/save_signature.php	Client Only	{"agreement_id": 1, "signature": "data:image/png;base64,..."} {"error": false, "message": "Agreement signed & stored"}	

6. Database Architecture & Complete 13-Table Schema Dictionary

6.1 Relational Architecture & Entity Relationships

The LogixPulse database engine utilizes a centralized relational schema designed to support both PostgreSQL and MySQL/MariaDB with zero dialect dissonance. The 13 tables are structured to maintain strict referential integrity through foreign keys with cascading rules:

Entity-Relationship Map (High-Level)

```
users (1) <----- (N) leads [assigned_to]
users (1) <----- (N) activity_logs [user_id]
leads (1) <----- (N) activity_logs [lead_id] (ON DELETE CASCADE)
users (1) <----- (N) projects [client_id] (ON DELETE CASCADE)
users (1) <----- (N) invoices [client_id] (ON DELETE CASCADE)
users (1) <----- (N) agreements [client_id] (ON DELETE CASCADE)
users (1) <----- (N) magic_codes [user_id] (ON DELETE CASCADE)
users (1) <----- (N) password_resets [user_id] (ON DELETE CASCADE)
users (1) <----- (1) client_timeline_progress [client_id] (ON DELETE CASCADE)
users (1) <----- (N) timeline_steps [client_id] (ON DELETE CASCADE)
columns_table (1) <- (N) tasks [column_id] (ON DELETE CASCADE)
```

6.2 Comprehensive Data Dictionary (All 13 Tables)

Table 1: `users` — Master Authentication & Identity

Column Name	Data Type	Nullable	Default	Description & Business Constraints
<code>id</code>	INT AUTO_INCREMENT	NO	PRIMARY KEY	Unique primary identifier.
<code>name</code>	VARCHAR(100)	NO	"	User full legal name.
<code>email</code>	VARCHAR(190)	NO	UNIQUE	Primary login identifier. Must be unique.
<code>password_hash</code>	VARCHAR(255)	NO	"	Native Bcrypt password hash generated via <code>password_hash()</code> .
<code>role</code>	VARCHAR(50)	NO	'client'	RBAC role: <code>admin</code> , <code>executive</code> , <code>head</code> , <code>team</code> , <code>client</code> .
<code>phone</code>	VARCHAR(50)	YES	NULL	Direct contact phone number.
<code>company</code>	VARCHAR(150)	YES	NULL	Organization or commercial client account name.
<code>status</code>	VARCHAR(20)	NO	'active'	Account state: <code>active</code> , <code>inactive</code> , <code>suspended</code> .
<code>created_at</code>	TIMESTAMP	NO	CURRENT_TIMESTAMP	Account creation timestamp.
<code>updated_at</code>	TIMESTAMP	NO	CURRENT_TIMESTAMP	Automatic modification timestamp.

Table 2: `leads` — Commercial Pipeline Prospects

Column Name	Data Type	Nullable	Default	Description & Business Constraints
<code>id</code>	INT AUTO_INCREMENT	NO	PRIMARY KEY	Unique lead identifier.
<code>name</code>	VARCHAR(200)	NO	"	Primary prospect contact name.
<code>first_name</code> / <code>last_name</code>	VARCHAR(100)	YES	"	Dual-schema decomposed name components.
<code>email</code>	VARCHAR(255)	YES	"	Lead email address. Validated server-side.
<code>phone</code>	VARCHAR(50)	YES	"	Lead phone number.
<code>company</code>	VARCHAR(255)	YES	"	Prospecting organization.
<code>source</code>	VARCHAR(100)	YES	"	Acquisition origin: Website, Referral, Inbound, LinkedIn.
<code>status</code>	VARCHAR(50)	NO	'new'	Canonical pipeline status: <code>new</code> , <code>contacted</code> , <code>meeting</code> , <code>closed</code> .
<code>stage</code>	VARCHAR(50)	NO	'new'	Dual-schema mirror column kept identical to <code>status</code> .
<code>assigned_to</code>	INT	YES	NULL	FK to <code>users(id)</code> . Assigned sales representative.
<code>notes</code>	TEXT	YES	NULL	Initial prospect notes captured during intake.
<code>created_at</code> / <code>updated_at</code>	TIMESTAMP	NO	CURRENT_TIMESTAMP	Lead ingestion and update audit timestamps.

Table 3: `activity_logs` — Historical Audit & Note Repository

Column Name	Data Type	Nullable	Default	Description & Business Constraints
<code>id</code>	BIGINT AUTO_INCREMENT	NO	PRIMARY KEY	High-capacity identifier for high-volume logs.
<code>lead_id</code>	INT	YES	NULL	FK to <code>leads(id)</code> with <code>ON DELETE CASCADE</code> . Indexed.
<code>user_id</code>	INT	YES	NULL	FK to <code>users(id)</code> with <code>ON DELETE SET NULL</code> . Indexed.
<code>action</code>	VARCHAR(255)	YES	NULL	Action code: <code>stage_change</code> , <code>add_note</code> , <code>lead_created</code> .
<code>activity_type</code>	VARCHAR(100)	YES	NULL	Category: <code>note</code> , <code>stage_change</code> , <code>system</code> .
<code>description</code>	TEXT	YES	NULL	Formatted summary ("Moved from Contacted to Meeting Booked").
<code>note</code>	TEXT	YES	NULL	Raw manual note text authored by sales representative.
<code>created_by</code>	VARCHAR(150)	YES	NULL	Cached author display name for historical immutability.
<code>created_at</code>	TIMESTAMP	NO	CURRENT_TIMESTAMP	Event occurrence timestamp. Indexed compoundly.

Table 4: `projects` — Client Project Engagements

Column Name	Data Type	Nullable	Default	Description & Business Constraints
<code>id</code>	INT AUTO_INCREMENT	NO	PRIMARY KEY	Unique engagement identifier.
<code>name</code>	VARCHAR(255)	NO	None	Project title (e.g., "Enterprise Cloud Migration").
<code>client_id</code>	INT	NO	None	FK to <code>users(id)</code> with <code>ON DELETE CASCADE</code> .
<code>status</code>	VARCHAR(50)	NO	'active'	State: <code>active</code> , <code>completed</code> , <code>on_hold</code> .
<code>start_date</code> / <code>end_date</code>	DATE	YES	NULL	Contractual schedule boundary dates.
<code>description</code>	TEXT	YES	NULL	Project scope overview.

Table 5: `invoices` — Commercial Invoices & Storage References

Column Name	Data Type	Nullable	Default	Description & Business Constraints
<code>id</code>	BIGINT AUTO_INCREMENT	NO	PRIMARY KEY	Unique invoice identifier.
<code>invoice_number</code>	VARCHAR(50)	YES	NULL	Human-readable invoice code (e.g., "INV-2026-0042").
<code>client_id</code>	INT	NO	None	FK to <code>users(id)</code> . Mandatory tenant isolation binding.
<code>project_id</code>	INT	YES	NULL	FK to <code>projects(id)</code> . Optional project association.
<code>amount</code>	DECIMAL(12,2)	NO	0.00	Financial balance billed. Precision: 2 decimal places.
<code>status</code>	VARCHAR(50)	NO	'draft'	Billing state: <code>draft</code> , <code>unpaid</code> , <code>paid</code> , <code>overdue</code> .
<code>original_filename</code>	VARCHAR(255)	YES	NULL	Preserved upload filename for client streaming presentation.
<code>stored_filename</code>	VARCHAR(100)	YES	UNIQUE	Randomized safe storage filename in <code>uploads/invoices/</code> .
<code>issue_date</code> / <code>due_date</code>	DATE	YES	NULL	Billing lifecycle dates.

Table 6: `agreements` — Master Service Agreements & Signatures

Column Name	Data Type	Nullable	Default	Description & Business Constraints
<code>id</code>	INT AUTO_INCREMENT	NO	PRIMARY KEY	Unique agreement record identifier.
<code>client_id</code>	INT	NO	None	FK to <code>users(id)</code> . Client party to the contract.
<code>title</code>	VARCHAR(255)	NO	None	Agreement title ("Master Service Agreement - 2026").
<code>content</code>	TEXT	YES	NULL	Legal clause body text.
<code>status</code>	VARCHAR(50)	NO	'draft'	Contract state: <code>draft</code> , <code>sent</code> , <code>signed</code> , <code>rejected</code> .
<code>signed_date</code>	DATE	YES	NULL	Execution date populated upon signature ingestion.

Table 7: magic_codes — Passwordless OTP Tokens

Column Name	Data Type	Nullable	Default	Description & Business Constraints
id	BIGINT AUTO_INCREMENT	NO	PRIMARY KEY	Unique token identifier.
user_id	INT	YES	NULL	FK to users(id) . Optional user reference.
email	VARCHAR(190)	YES	NULL	Target email address. Indexed for rapid lookups.
code	VARCHAR(10)	YES	NULL	Cleartext 6-digit OTP (or token hash).
expires_at	DATETIME	NO	None	Expiration threshold (10 minutes post generation).
attempts	INT UNSIGNED	NO	0	Failed submission counter. Throttles at 5 attempts.
used_at	DATETIME	YES	NULL	Timestamp of successful authentication. Single-use guard.

Table 8: password_resets — Cryptographic Recovery Tokens

Column Name	Data Type	Nullable	Default	Description & Business Constraints
id	BIGINT AUTO_INCREMENT	NO	PRIMARY KEY	Unique reset token identifier.
user_id	INT	YES	NULL	FK to users(id) . Account to be reset.
email	VARCHAR(190)	YES	NULL	Target account email address.
token_hash	CHAR(64)	YES	NULL	SHA-256 hash of the 64-character entropy reset token.
expires_at	DATETIME	NO	None	Expiration threshold (60 minutes post generation).
used_at	DATETIME	YES	NULL	Timestamp of password modification. Single-use guard.

Tables 9 & 10: columns_table & tasks — Kanban Structure

Table	Column Name	Data Type	Description
columns_table	id , name , position	INT, VARCHAR(100), INT	Configurable Kanban column definitions and visual ordering.
tasks	id , title , column_id , position	INT, VARCHAR(255), INT, INT	Tasks mapped to columns with foreign key cascading.

Tables 11 & 12: `client_timeline_progress` & `timeline_steps` — Onboarding

Table	Column Name	Data Type	Description
<code>client_timeline_progress</code>	<code>client_id</code> , <code>current_step_order</code>	INT (PK, FK), INT	Stores active milestone order for client onboarding.
<code>timeline_steps</code>	<code>id</code> , <code>client_id</code> , <code>step_order</code> , <code>title</code> , <code>date_label</code>	INT, INT (FK), INT, VARCHAR(150), VARCHAR(50)	Per-client customized milestone steps and timeline labels.

Table 13: `internal_users` & `clients` — Legacy Compatibility Tables

Dedicated standalone tables preserved to guarantee backward compatibility with early Week 7 developer test forms, featuring columns: `id` , `name` , `email` , `password` , `role` , `status` , and `created_at` .

7. Non-Functional Requirements (NFRs)

7.1 Security Requirements & OWASP ASVS Mitigation Matrix

OWASP Threat Category	LogixPulse Architectural Vulnerability Risk	Mandatory Engineering Defense & Verification Rule
A01: Broken Access Control (BOLA / IDOR)	Client manipulates URL parameter (<code>download.php?id=2</code>) to access another company's financial invoice or timeline.	Tenant identity MUST be extracted strictly from <code>\$_SESSION['user_id']</code> . Database queries verify <code>invoices.client_id == \$_SESSION['user_id']</code> . Violations immediately terminate with HTTP 403.
A02: Cryptographic Failures	Storage of plaintext passwords or weak MD5/SHA1 hashes susceptible to rainbow table breaches.	Passwords MUST be hashed using PHP's native <code>password_hash(\$pwd, PASSWORD_DEFAULT)</code> (Bcrypt) and verified via <code>password_verify()</code> . Demo seed scripts must contain pre-computed verified hashes.
A03: Injection (SQL Injection)	User input injected into search strings or stage updates altering query semantics.	100% of SQL executions MUST use PDO parameterized prepared statements (<code>\$pdo->prepare()</code> and <code>\$stmt->execute([\$. . .])</code>). Zero string concatenation (<code>"SELECT ... "</code> . <code>\$input</code>) permitted in codebase.
A04: Insecure Design & RCE	Malicious actor uploads <code>shell.php</code> disguised as an invoice to execute code on the server.	Uploads directory is shielded with <code>.htaccess</code> disabling script execution. Invoices are validated via <code>finfo_file()</code> for true <code>application/pdf</code> MIME, and stored under randomized UUID filenames.
A07: Identification & Auth Failures	Session fixation or brute forcing of 6-digit magic codes.	Calls <code>session_regenerate_id(true)</code> upon every authentication event. Magic codes expire in 10 minutes and lock after 5 failed attempts. Single-use enforcement via <code>used_at</code> timestamp.

7.2 Performance & Scalability Requirements

- **API Response Latency:** 95% of API requests (excluding binary file downloads) MUST return in under **200 milliseconds** under nominal load (50 concurrent users).
- **Kanban Drag Responsiveness:** Client-side UI must transition the dragged card instantly (<16ms frame budget), executing the background persistence request asynchronously.
- **Database Indexing:** Compound indexes on (`lead_id`, `created_at`, `id`) in `activity_logs` and `client_id` in `invoices` guarantee sub-10ms lookup times even as tables scale past 50,000 records.
- **Database Connection Management:** The application implements a singleton connection pattern in `config/database.php` , ensuring exactly one PDO connection per HTTP request lifecycle.

7.3 Usability, Accessibility & Responsive Layout Standards

- **Design Fidelity:** Frontend implementations by Squads FE-A and FE-B MUST conform precisely to the pre-approved LogixPulse design system tokens, typography scales, and component layouts.

-
- **Mobile Viewport Support:** All screens must remain fully functional and visually unbroken on mobile screens down to **320px width**. Specifically:
 - Kanban columns gracefully stack or provide horizontal scrolling.
 - HTML5 Canvas signature pad adjusts dynamically to available screen width without clipping drawing strokes.
 - Data tables (Invoices, Leaderboard) provide responsive card transformations or clean horizontal scrolling.
 - **Real-Time Visual Feedback:** Every asynchronous action lasting >300ms must display a loading spinner. Every mutation must trigger a color-coded toast notification (Green for success, Red for failure).

7.4 Maintainability, Code Quality & Clean Architecture

- **Zero Unhandled Fatal Errors:** All database and file I/O operations must be wrapped in `try/catch` blocks. Under no circumstances may raw PHP call stacks or fatal errors be exposed to the client.
- **Strict `exit;` Termination:** Every API endpoint MUST invoke `exit;` or `die()` immediately following an error JSON response to prevent PHP from executing subsequent code blocks.
- **Single Source of Truth:** Database configuration must reside exclusively in `config/database.php` reading from environment variables (`.env`). Legacy configuration scripts act as thin wrappers forwarding to this unified singleton.

8. Verification Protocol, Acceptance Criteria & Traceability Matrix

8.1 The Daily Acceptance Rule & Evaluation Rubric

Sprint tasks are evaluated daily at 6:30 PM (with final submission cutoff at 9:00 PM PKT). A task is certified complete if and only if it satisfies all three gates of the **Daily Evaluation Rubric**:

1. **Live Functional Execution**: Does the feature actually work when tested live in a browser/Postman (not merely "looks right")?
2. **Verifiable Proof of Work**: Does the submitted screencast, screenshot, or pull request genuinely demonstrate the acceptance criteria without simulation?
3. **Edge Case Resilience**: Does the code survive invalid input, unauthorized roles, and empty states without throwing PHP fatal errors?

8.2 Requirements Traceability Matrix (RTM)

Requirement ID	Functional Requirement Summary	Sprint Task Mapping	Owning Squad	Verification Method
FR-AUTH-01	Traditional Bcrypt Authentication	BE-W7D1-3 , FEA-W7D2-1	Squad BE / FE-A	Live Login Test with Password@123
FR-AUTH-02	Passwordless Magic Code OTP	BE-W7D2-3 , FEB-W7D2-1	Squad BE / FE-B	OTP Generation & Verification Flow
FR-AUTH-03	Self-Service Password Reset	BE-W7D2-3 , FEB-W7D1-2	Squad BE / FE-B	Token Issuance & Password Update
FR-AUTH-04	Session Router & Identity Check	BE-W7D2-2 , FEB-W7D1-3	Squad BE / Router	Role Redirection & who_is_logged_in
FR-CRM-01	4-Column Kanban Visual Board	FEA-W7D1-3 , BE-W7D3-2	Squad FE-A / BE	Database Lead Hydration Test
FR-CRM-02	Drag-and-Drop Card Stage Move	FEA-W7D4-1 , FEA-W7D4-2	Squad FE-A	Drag Card & Hard Page Refresh
FR-CRM-03	Transactional Stage Persistence	BE-W7D4-1	Squad BE	Database Status & Log Verification
FR-CRM-04	Add Lead Modal & Server Validation	FEA-W7D3-1 , BE-W7D3-1	Squad FE-A / BE	Form Input & SQL Injection Tests
FR-CRM-05	Real-Time Lead Search Bar	FEA-W7D5-1 , BE-W7D5-1	Squad FE-A / BE	Partial String Debounced Filtering
FR-CRM-06	Lead Details Offcanvas Drawer	FEA-W7D3-3 , BE-W7D4-2	Squad FE-A / BE	Card Click & Metadata Inspection
FR-LOG-01	Manual Free-Text Notes	FEA-W7D5-2 , BE-W7D5-2	Squad FE-A / BE	Note Authoring & Timeline Render
FR-KPI-01	Executive Dashboard KPI Cards	BE-W7D5-4 , FEA-W7D5-3	Squad BE / FE-A	Metric Delta Calculation vs SQL
FR-KPI-02	Pipeline Overview & Velocity	BE-W8D1-4 , FEA-W8D1-4	Squad BE / FE-A	Conversion & Velocity Verification
FR-LOG-03	Recent Activity Live Stream	BE-W8D2-4 , FEA-W8D2-4	Squad BE / FE-A	Live Action Cross-Feed Test
FR-KPI-03	Team Leaderboard & Quota Engine	BE-W8D3-4 , FEA-W8D3-4	Squad BE / FE-A	Territory Filtering & Quota Badges
FR-CLT-01	Client Onboarding Timeline	FEB-W7D3-1 , BE-W7D3-3	Squad FE-B / BE	Multi-Tenant Dynamic Phase Check
FR-AGR-02	HTML5 Canvas E-Signature Pad	FEB-W7D4-2	Squad FE-B	Draw, Clear & Touch Input Test
FR-AGR-03	Signature Ingestion & PNG Storage	BE-W8D1-1 , FEB-W8D1-1	Squad BE / FE-B	Base64 Decode & File Creation

Requirement ID	Functional Requirement Summary	Sprint Task Mapping	Owning Squad	Verification Method
FR-INV-01	Admin Protected Invoice Upload	BE-W7D4-3 , FEB-W7D2-3	Squad BE	PDF MIME & 10MB Ceiling Validation
FR-INV-02	Client-Isolated PDF Download	FEB-W7D4-1 , BE-W8D1-2	Squad FE-B / BE	Cross-Tenant IDOR Download Attack
FR-INV-04	Upload Hardening & .htaccess	BE-W8D2-2	Squad BE	Direct PHP Execution Block Test

8.3 Demo Day Exit Gates

- **Demo Day #1 (End of Week 7):** Full CRM Kanban board and Client Portal running entirely off live database records. Verified lead creation, drag-and-drop persistence, multi-column search, invoice listing, and signature canvas drawing.
- **Demo Day #2 (End of Week 8) — Module 1 Production Exit:** Complete, unbroken end-to-end flow demonstrated live without manual database intervention:
 1. BE Squad presents live database tables and schema relationships in pgAdmin/MySQL.
 2. Executive Dashboard walked through live: KPI cards, pipeline velocity, recent activity, and team leaderboard.
 3. BD Representative logs in, creates lead, moves card across board, and authors qualitative note.
 4. Admin logs in and safely deletes test lead via confirmation modal.
 5. Client logs in via traditional password and magic code OTP, verifies milestone timeline, downloads isolated invoice PDF, and executes digital Master Agreement.
 6. Security verification: Live demonstration of 403 Forbidden rejection when accessing another tenant's invoice or administrative actions.

9. Sprint Governance, Squad Rosters & Incident Retrospective

9.1 Sprint 1 PHP Practice Squad Rosters

Squad Identifier	Intern ID	Intern Name	Assigned Role & Responsibilities
Squad PHP-BE (Backend & Database)	DL-IN-0111	Zainab Batool	Squad Lead · Architecture, Schema, Auth, Stage Transactions, Security Audits
	DL-IN-0091	Atiqa Maqbool	DB Connection Singleton, Lead Search Engine, Activity History, KPI Endpoints
	DL-IN-0078	Areesha Sarwar	Traditional Login Checker, Save Lead Endpoint, Secure Isolated Invoice Storage
Squad PHP-FE-A (CRM Board & Executive)	DL-IN-0093	Fatima Nadeem	Squad Lead · Page Frame Shell, Add Lead Form, Demo Script, Lead Deletion
	DL-IN-0069	Minahil Imran	Login Screen, Card Visual Styling, HTML5 Drag-and-Drop Implementation
	DL-IN-0074	Muskan Arshad	Kanban Layout, Board Real-Data Hydration, Drag Persistence Integration
	DL-IN-0086	Momna Hafeez	Sidebar Navigation, Details Slide Drawer, KPI Metric Cards, Activity Panel
Squad PHP-FE-B (Client Portal & Contract)	DL-IN-0090	Momna Munir	Squad Lead · Auth Screens Hookup, HTML5 Canvas Signature Pad, Demo Script
	DL-IN-0076	Sayyda Arooj	Password Reset Screens, Timeline Visualization, Invoice Streaming Integration
	DL-IN-0070	Kaneez Fatima	Client Dashboard Shell, Real Timeline Data Integration, Mobile Responsiveness
	DL-IN-0089	Halima Sadia	Traditional & Magic Login Screens, Invoice Table, Agreement Presentation Shell

9.2 Rules of Engagement (RoE) & Non-Negotiables

- Hard Cutoff at 9:00 PM PKT:** Daily proof of work MUST be submitted and verified before 9:00 PM. A submission at 9:01 PM is logged as incomplete.
- 45-Minute Escalation Rule:** An engineer stuck on a blocking issue for >45 minutes MUST escalate immediately to their Squad Lead. Silent spinning is prohibited.
- Server-Side Primacy:** Client-side JavaScript validation is strictly a UX convenience. Every single constraint (non-empty, email format, MIME type, authorization) MUST be repeated and enforced server-side in PHP.
- No Scope-Shrinking:** If a task cannot be completed within its designated time box, it must be flagged before 3:00 PM rather than silently truncated or dropped.

9.3 Architectural Incident Retrospective & Prevention Standards

Midway through Sprint 1, an architectural review identified critical integration anomalies arising from uncoordinated parallel development branches. These incidents and their permanent preventative standards are codified below:

Identified Architectural Incident	Root Cause Analysis	Codified Engineering Prevention Rule
Configuration File Sprawl (5 distinct database files pointing to 4 different DB names)	Squads created independent local connection files (<code>db_connect.php</code> , <code>config.php</code> , <code>database.php</code>) during parallel development.	Rule: Single Source of Truth. All scripts MUST include <code>config/database.php</code> . Legacy connection files are strictly backward-compatible wrappers forwarding to the PDO singleton.
Routing Collision at Root (Opening localhost served empty placeholder instead of login)	Default web servers prioritize <code>index.php</code> over <code>index.html</code> . A ticket committed a dummy dashboard as <code>index.php</code> .	Rule: Session Router Architecture. <code>index.php</code> serves as an intelligent traffic dispatcher routing authenticated users based on role and unauthenticated users to <code>index.html</code> .
Corrupted Bcrypt Hashes (Demo accounts locked out due to dummy string hashes)	Seed files inserted literal placeholder strings (<code>\$2y\$10\$...1JZ3JZ3J3e</code>) instead of genuine hashes.	Rule: Verified Hash Seeding. All database seed files MUST generate hashes via <code>password_hash('Password@123' , PASSWORD_DEFAULT)</code> tested via <code>password_verify()</code> before commit.
Dual Schema Drift (Code expecting <code>leads.status</code> while updates targeted <code>leads.stage</code>)	FE-A and BE squads utilized diverging column naming conventions across tickets.	Rule: Dual-Schema Compatibility Layer. SQL queries update both <code>status</code> and <code>stage</code> columns simultaneously to maintain 100% interoperability across all legacy and modern modules.

10. Appendices & Formal Sign-Off

10.1 Quick Reference & Troubleshooting Guide

Symptom	Root Cause Diagnosis	Immediate Resolution Step
"Database connection failed"	PostgreSQL VPS unreachable or local MySQL service stopped.	Verify MySQL is running on <code>127.0.0.1:3306</code> ; verify <code>.env</code> credentials; client portal automatically engages in-memory mock mode for standalone testing.
"Login failed with Password@123"	Stale seed data or unverified password hashes.	Re-import <code>schema.sql</code> or run <code>sql/03_seed_data.sql</code> , which contains pre-verified Bcrypt hashes for <code>Password@123</code> .
"Drag and drop does not persist"	User session expired or backend stage update returned error.	Open browser developer console → inspect network tab for <code>update_lead_stage.php</code> → ensure user is authenticated with non-client role.
"Invoice download yields 403"	Strict tenant isolation active; invoice does not belong to logged-in client.	Confirm <code>invoice.client_id</code> matches active session user ID; verify user is logged in as the owning client account or as admin.

10.2 Formal Document Review & Sign-Off

This Software Requirements Specification for LogixPulse Module 1 (CRM & Client Onboarding) has been thoroughly reviewed and validated against all Sprint 1 deliverables, technical guides, and architectural security standards. By signing below, the engineering leadership certifies this document as the formal production baseline.

David Chen

Technical Head & Chief Architect
DevLogix Technology · Engineering Track
[\[APPROVED & CERTIFIED - SEP 25, 2026\]](#)

Practice Manager

PHP Practice Lead & SIP'26 Program Director
DevLogix Strategic Internship Program
[\[APPROVED & CERTIFIED - SEP 25, 2026\]](#)

Abdullah Bin Masood · Alishba Amin

Line Management Oversight
Software Engineering Operations
[\[VERIFIED SPRINT BASELINE - SEP 25, 2026\]](#)

Zainab Batool · Fatima Nadeem · Momna Munir

Squad Leads (PHP-BE, PHP-FE-A, PHP-FE-B)
Module 1 Core Engineering Squads
[\[ACCEPTED BY SQUAD LEADS - SEP 25, 2026\]](#)

*** END OF SOFTWARE REQUIREMENTS SPECIFICATION — LOGIXPULSE MODULE 1 ***